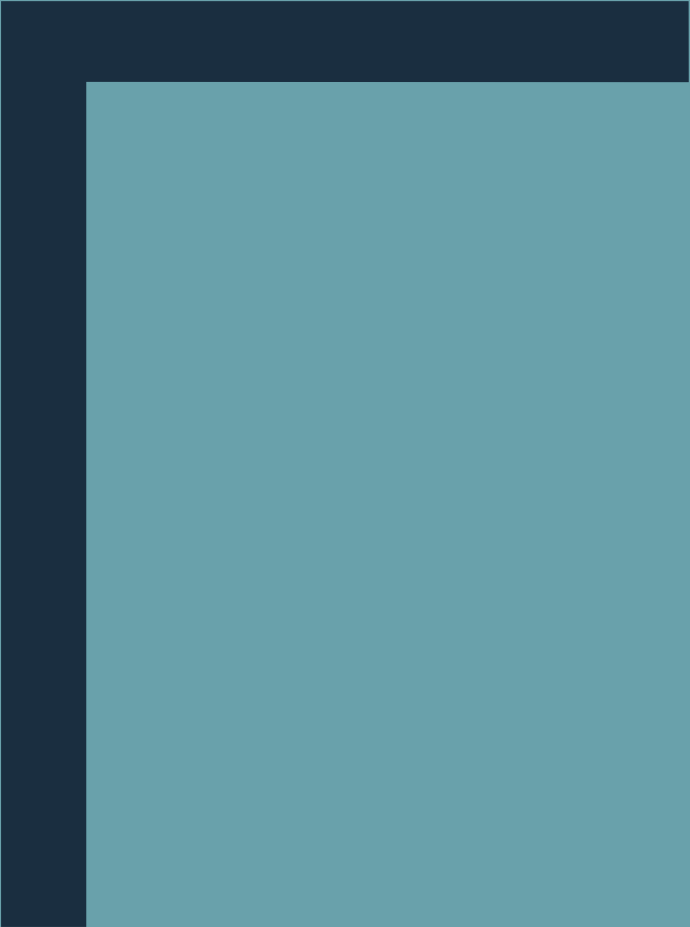
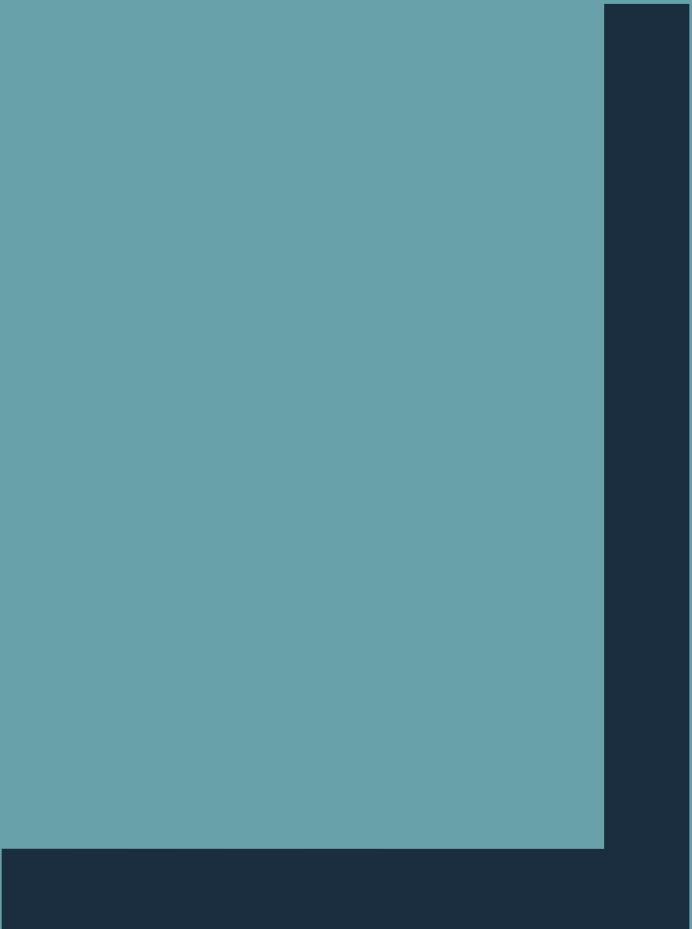




ARRAYS II



Static Local Arrays and Automatic Local Arrays

- A static local variable exists for the duration of the program but is visible only in the function body.
- We can apply static to a local array definition so the array is not created and initialized each time the function is called and the array is not destroyed each time the function is exited in the program.
- This reduces program execution time, particularly for programs with frequently called functions that contain large arrays.

Static Local Arrays and Automatic Local Arrays

- Function `staticArrayInit` is called twice.
- The local static array in the function is initialized to zero before program startup.
- The function prints the array, adds 5 to each element and prints the array again.
- The second time the function is called, the static array contains the values stored during the first function call.



```
#include<stdio.h>
```

```
void staticArrayInit(void);
```

```
void automaticArrayInit(void);
```

```
int main(void)
```

```
{
```

```
    printf("First call to each function:\n");
```

```
    staticArrayInit();
```

```
    automaticArrayInit();
```

```
    printf("Second call to each function:\n");
```

```
    staticArrayInit();
```

```
    automaticArrayInit();
```

```
    return 0;
```

```
}
```



```
void staticArrayInit(void)
{
    static int array1[3];

    printf("\nValues on entering staticArrayInit:\n");

    for (int i = 0; i < 3; i++)
    {
        printf("array1[%d]=%d ", i, array1[i]);
    }
    printf("\nValues on exiting staticArrayInit:\n");
    for (int i = 0; i < 3; i++)
    {
        printf("array1[%d]=%d ", i, array1[i]+=5);
    }
}
```



```
void automaticArrayInit(void)
{
    int array2[3] = {1,2,3};

    printf("\nValues on entering automaticArrayInit:\n");

    for (int i = 0; i < 3; i++)
    {
        printf("array2[%d]=%d ", i, array2[i]);
    }
    printf("\nValues on exiting automaticArrayInit:\n");
    for (int i = 0; i < 3; i++)
    {
        printf("array2[%d]=%d ", i, array2[i] += 5);
    }
}
```

First call to each function:

Values on entering staticArrayInit:

array1[0]=0 array1[1]=0 array1[2]=0

Values on exiting staticArrayInit:

array1[0]=5 array1[1]=5 array1[2]=5

Values on entering automaticArrayInit:

array2[0]=1 array2[1]=2 array2[2]=3

Values on exiting automaticArrayInit:

array2[0]=6 array2[1]=7 array2[2]=8

Second call to each function:

Values on entering staticArrayInit:

array1[0]=5 array1[1]=5 array1[2]=5

Values on exiting staticArrayInit:

array1[0]=10 array1[1]=10 array1[2]=10

Values on entering automaticArrayInit:

array2[0]=1 array2[1]=2 array2[2]=3

Values on exiting automaticArrayInit:

array2[0]=6 array2[1]=7 array2[2]=8

Static Local Arrays and Automatic Local Arrays

- Arrays that are static are initialized once at program startup.
- If you do not explicitly initialize a static array, that array's elements are initialized to zero by default.
- Next demonstrates function `staticArrayInit` with a local static array and function `automaticArrayInit` with a local automatic array.

Static Local Arrays and Automatic Local Arrays

- Function `automaticArrayInit` is also called twice.
- The elements of the automatic local array in the function are initialized with the values 1, 2 and 3.
- The function prints the array, adds 5 to each element and prints the array again.
- The second time the function is called, the array elements are initialized to 1, 2 and 3 again because the array has automatic storage duration.

Passing Arrays to Functions

- To pass an array argument to a function, specify the array's name without any brackets.
- For example, if array *hourlyTemperatures* has been defined as

int hourlyTemperatures[24];

the function call

modifyArray(hourlyTemperatures, 24);

passes array *hourlyTemperatures* and its size to function *modifyArray*.

Passing Arrays to Functions

- C automatically passes arrays to functions by reference — the called functions can modify the element values in the callers' original arrays.
- The name of the array evaluates to the address of the first element of the array.
- Because the starting address of the array is passed, the called function knows precisely where the array is stored.
- Therefore, when the called function modifies array elements in its function body, it's modifying the actual elements of the array in their original memory locations.

Passing Arrays to Functions

- Although entire arrays are passed by reference, individual array elements are passed by value exactly as simple variables are.
- To pass an element of an array to a function, use the indexed name of the array element as an argument in the function call.

Passing Arrays to Functions

- For a function to receive an array through a function call, the function's parameter list must specify that an array will be received.
- For example, the function header for function `modifyArray` might be written as

```
void modifyArray(int b[ ], int size)
```

indicating that `modifyArray` expects to receive an array of integers in parameter `b` and the number of array elements in parameter `size`.

- The size of the array is not required between the array brackets.

```
#include <stdio.h>
void modifyArray(int b[], int size)
{
    for (int j = 0; j < size; ++j)
        b[j] *= 2;
}
void modifyElement(int e)
{
    printf("Value in modifyElement is %d\n", e *= 2);
}
int main(void)
{
    int a[5] = { 0, 1, 2, 3, 4 };
    printf("The values of the original array are:");
    for (int i = 0; i < 5; i++)
        printf("%3d", a[i]);
    modifyArray(a, 5);
    printf("\nThe values of the modified array are:");
    for (int i = 0; i < 5; i++)
        printf("%3d", a[i]);
    printf("\nThe value of a[3] is %d\n", a[3]);
    modifyElement(a[3]);
    printf("The value of a[3] is %d\n", a[3]);
}
```

Passing Arrays to Functions

OUTPUT:

The values of the original array are: 0 1 2 3 4

The values of the modified array are: 0 2 4 6 8

The value of a[3] is 6

Value in modifyElement is 12

The value of a[3] is 6

Multidimensional Arrays

- Arrays in C can have multiple indices.
- A common use of multidimensional arrays is to represent tables of values consisting of information arranged in rows and columns.
- To identify a particular table element, we must specify two indices: The first (by convention) identifies the element's row and the second (by convention) identifies the element's column.
- Multidimensional arrays can have more than two indices.
- Tables or arrays that require two indices to identify a particular element are called two-dimensional arrays.

Multidimensional Arrays

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Diagram illustrating the indexing of a 2D array. The array is represented as a grid of elements. The first index (left) is the Row index, and the second index (right) is the Column index. The array name 'a' is shown in the first part of each element address.

Legend:

- Column index
- Row index
- Array name

Multidimensional Arrays

- Figure illustrates a two-dimensional array, a .
- The array contains three rows and four columns, so it's said to be a 3-by-4 array.
- In general, an array with m rows and n columns is called an m -by- n array

Multidimensional Arrays

- Every element in array **a** is identified in Figure by an element name of the form **a[i][j]**; **a** is the name of the array, and **i** and **j** are the indices that uniquely identify each element in **a**.
- The names of the elements in row 0 all have a first index of 0; the names of the elements in column 3 all have a second index of 3.

Multidimensional Arrays

- A multidimensional array can be initialized when it's defined, much like a one-dimensional array.
- For example, a two-dimensional array `int b[2][2]` could be defined and initialized with

```
int b[2][2] = {{1, 2}, {3, 4}};
```

- The values are grouped by row in braces.
- The values in the first set of braces initialize row 0 and the values in the second set of braces initialize row 1.
- So, the values 1 and 2 initialize elements `b[0][0]` and `b[0][1]`, respectively, and the values 3 and 4 initialize elements `b[1][0]` and `b[1][1]`, respectively.

Multidimensional Arrays

- *If there are not enough initializers for a given row, the remaining elements of that row are initialized to 0.*

- Thus,

- `int b[2][2] = {{1}, {3, 4}};`

would initialize `b[0][0]` to 1, `b[0][1]` to 0, `b[1][0]` to 3 and `b[1][1]` to 4.



```
#include <stdio.h>
int main(void)
{
    int array1[2][3] = { { 1, 2, 3 }, { 4, 5, 6 } };
    printf ("Values in array1 by row are:");
    printArray(array1);

    int array2[2][3] = { 1, 2, 3, 4, 5 };
    printf ("Values in array2 by row are:");
    printArray(array2);

    int array3[2][3] = { { 1, 2 }, { 4 } };
    printf ("Values in array3 by row are:");
    printArray(array3);
}
```

Multidimensional Arrays

Values in array1 by row are:

1 2 3

4 5 6

Values in array2 by row are:

1 2 3

4 5 0

Values in array3 by row are:

1 2 0

4 0 0